

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058, India
(Autonomous Institute Affiliated to University of Mumbai)

DEPARTMENT OF COMPUTER ENGINEERING

Generative AI Lab — AY 2026–27

EXPERIMENT 7

Domain-Specific Generative AI Chatbot using a Pre-trained Language Model and Streamlit

UID	2023300139
Name	Raj Kalpesh Mathuria
Batch	PE-C
Course	Generative AI (PE III)
Course Code	CE413
Academic Year	2026–27
Domain	Banking Customer Support

Experiment report prepared according to the prescribed Generative AI Lab Manual AY 2026–27.

Github Repo Link: <https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Exp7>

1. Introduction

This experiment implements a domain-specific Generative AI chatbot for Banking Customer Support using a pre-trained language model, prompt engineering, domain-specific information, and a Streamlit interface. The same banking domain used in the preceding LoRA/Fine-Tuning experiment is retained so the two approaches can be compared.

2. Aim

To design and implement a domain-specific Generative AI chatbot using a pre-trained language model, prompt engineering, domain-specific knowledge, and a Streamlit-based user interface.

3. Objectives

The experiment aims to understand an LLM chatbot architecture; use a pre-trained language model; provide domain information through prompting; design a system prompt; build a Streamlit interface; connect Python to an inference service; evaluate responses; identify hallucination and incomplete knowledge; understand Fine-Tuning, Prompting and RAG; and convert an LLM into a user-facing application.

4. Theoretical Background

A Generative AI chatbot uses an LLM to understand a user's question and generate a natural-language response. In this experiment, domain adaptation is achieved at inference time by placing banking information and instructions in the prompt; model parameters are not modified. Prompt engineering is used to constrain the model to the supplied domain. Hallucination refers to unsupported or fabricated claims generated by a model. The experiment demonstrates that stronger prompts can improve scope handling but cannot guarantee elimination of unsupported generation.

5. System Architecture

The application follows: User Question → Streamlit UI → Prompt Builder → System Instructions + Banking Domain Information → Pre-trained LLM → Generated Response → Streamlit UI. Streamlit provides the user-facing interface, while the prompt builder supplies the context used by the hosted model.

6. Implementation

The project is modular, with app.py for the Streamlit application, config.yaml for configuration, data/domain_information.yaml for banking knowledge, data/evaluation_questions.yaml for evaluation questions, and src modules for configuration, domain loading, prompt construction, LLM inference, and evaluation. The final configured model was openai/gpt-oss-120b, accessed through Hugging Face InferenceClient. Generation used max_new_tokens=200 and temperature=0.2.

7. Domain Information

The synthetic Banking Customer Support knowledge base covers password/PIN reset, lost or stolen debit cards, ATM cash issues, unauthorized transactions, statements, beneficiaries, declined card payments, locked accounts, transfers, transaction notifications, and five additional facts added for Modification 2: monthly statement frequency, registered contact information, credential security, transfer verification, and support escalation.

8. Modification 1 — Prompt Engineering

The domain prompt was strengthened so that the supplied domain information became the authoritative source for banking-specific claims. The prompt explicitly prohibited invented policies, procedures, fees, limits, timeframes and requirements; instructed the model not to use general knowledge to fill gaps; required explicit handling of unavailable information; and required multi-part questions to be addressed separately.

Q	Observed response	Assessment
1	Beneficiary can be added through online banking and additional verification may be required.	Supported by supplied domain information.
2	Unauthorized transaction should be reported immediately and the affected card/account access may be blocked.	Supported by supplied domain information.
3	Beneficiary-management procedure was given; exact required details were acknowledged as unavailable.	Partially supported; missing details were handled correctly.

Q	Observed response	Assessment
4	Capital of France was identified as outside the supplied banking scope.	Correct out-of-domain handling.
5	CEO of Microsoft was identified as outside the supplied banking scope.	Correct out-of-domain handling.

Result: the stricter prompt successfully preserved domain boundaries and improved explicit acknowledgement of unavailable information. The observed banking claims in these five tests were supported by the supplied domain data.

9. Modification 2 — Additional Domain Facts

Five additional facts were added: monthly account statements; keeping the registered mobile number and email updated; never sharing the card PIN, online-banking password or one-time verification code; verifying recipient account details before transfers; and supplying transaction reference plus relevant date/time details when an unresolved issue is escalated.

Q	Observed response	Assessment
1	Monthly account statements can be requested through the online banking portal.	Correctly used new domain fact.
2	Registered mobile phone number and email address should be kept updated.	Correctly used new domain fact.
3	Card PIN, online-banking password and one-time verification code should not be shared.	Correctly used new domain fact.
4	The model instructed the user to verify the recipient account; the captured output ended before the second part.	Partially observable; first part supported, second part cannot be assessed from captured output.
5	Transaction reference and relevant date/time details should be provided to support.	Correctly used new domain fact.

Result: the chatbot successfully incorporated the newly supplied domain facts in the captured responses. The multi-part transfer test produced only a partial displayed response, so its second component is not claimed as verified.

10. Baseline Evaluation

Before Modification 1, ten questions were tested. The baseline generated generic procedures for several banking questions, while it correctly recognized that a specific domestic transfer timeframe was not contained in the supplied information and correctly rejected the two out-of-domain questions. These observations motivated the stricter prompt used in Modification 1.

Category	Observed baseline behavior
Beneficiary	Generic beneficiary-management steps were generated.
Password reset	A generic reset procedure was generated.
Unauthorized transaction	Generic reporting and blocking guidance was generated.
Transfer timeframe	The model correctly stated that a specific timeframe was not supplied.
Paraphrased/multi-part banking questions	Generic procedural guidance was generated.
Out-of-domain questions	The capital-of-France and Microsoft CEO questions were correctly rejected.

11. Comparison with LoRA/Fine-Tuning and RAG

Aspect	LoRA / Fine-Tuning	Experiment 7	RAG
Core idea	Adapt model using training.	Prompt + domain information.	Retrieve relevant external information before generation.
Parameters	Adapted during training.	Not modified.	Usually not modified for retrieval.
Knowledge	Encoded through adaptation/training data.	Supplied directly in prompt.	Retrieved from documents/knowledge base.
Retriever	Not required.	Not required.	Central component.
Scale	Useful for model adaptation.	Limited by prompt/context size.	Better suited to large external knowledge bases.

12. Analysis Questions

Q1. What is a Generative AI chatbot?

An LLM-powered application that understands user input and dynamically generates natural-language responses.

Q2. What is the role of the pre-trained LLM?

It performs language understanding and response generation without being fine-tuned in this experiment.

Q3. Why use the same domain as the previous experiment?

It allows the prompt-based approach to be compared conceptually with LoRA/Fine-Tuning using the same application domain.

Q4. How is this different from LoRA fine-tuning?

LoRA adapts model parameters through training; Experiment 7 supplies instructions and domain context at inference time.

Q5. Does this experiment modify LLM parameters?

No.

Q6. What is prompt engineering?

The deliberate design of instructions and context to guide an LLM's behavior.

Q7. Why include domain information in the prompt?

To provide the application-specific facts the model should use while generating answers.

Q8. What happens for information not present in the supplied domain?

The intended behavior is to state that the information is unavailable rather than invent it; the tests also show that prompting is not a perfect guarantee.

Q9. What is hallucination?

An unsupported or fabricated claim generated by a language model.

Q10. Can domain information completely eliminate hallucination?

No. It can improve grounding and scope control, but it cannot guarantee that every generated statement is supported.

Q11. Limitations of large prompt-based domain information?

Large prompts consume context, are costly to send repeatedly, are harder to maintain, and may exceed context limits.

Q12. How improve the chatbot for thousands of documents?

Use retrieval-based methods such as RAG to select relevant documents or chunks for each question.

Q13. Difference from RAG?

Experiment 7 supplies domain information directly in the prompt; RAG adds retrieval of relevant external knowledge before generation.

Q14. Why is RAG useful for a large knowledge base?

It avoids inserting the entire knowledge base into every prompt and instead supplies relevant retrieved information.

Q15. Role of Streamlit?

It provides the web interface for entering questions, displaying responses, continuing conversations and supporting application controls such as history.

13. Results and Discussion

The implementation successfully demonstrated a hosted pre-trained LLM chatbot with synthetic banking domain knowledge. Modification 1 improved the explicit domain-boundary behavior and acknowledgement of missing information. Modification 2 showed that newly supplied banking facts were directly reflected in the generated answers. The captured multi-part transfer

response was truncated, so only its visible portion is treated as verified.

The experiment also illustrates the central limitation of prompt-based domain adaptation: a model can be instructed to use supplied information, but prompting alone does not create a permanently trained domain expert or guarantee perfect factual grounding. For larger collections of documents, the manual's progression toward RAG provides a retrieval-based solution.

14. Security and Deployment

The Hugging Face access token must not be committed to source code or uploaded to GitHub. Secure environment variables or Streamlit secrets should be used for deployment. The local application used an environment variable for the inference token.

15. Conclusion

A Banking Customer Support Generative AI chatbot was successfully designed and implemented using a pre-trained language model, prompt engineering, domain-specific information and Streamlit. The experiment demonstrated how prompt design and supplied context influence LLM responses without modifying model parameters. The evaluations showed successful use of explicitly supplied facts and appropriate handling of out-of-domain questions, while also demonstrating the limitations of prompt-only grounding. These limitations motivate retrieval-augmented approaches such as RAG when the domain knowledge base becomes large.

Source Basis

Prepared using the Sardar Patel Institute of Technology Generative AI Lab Manual AY 2026–27, Experiment 7: Domain-Specific Generative AI Chatbot using a Pre-trained Language Model and Streamlit.